Capítulo IV - Análisis de Arquitectura y Métricas

Documento Técnico: Microservicio de Análisis Predictivo | Sistema de Gestión de Inventarios

FECHA	VERSIÓN	ALGORITMO	AUTOR
2026-10-21	1.0	Ridge Lag-4 (v1)	Ingeniería de Software

1. Reporte Técnico de Métricas de Entrenamiento y Validación

1.1 Arquitectura del Modelo

El microservicio implementa una **Regresión Ridge con autoregresión de 4 lags (ridge-lag4-v1)**. Su objetivo es proyectar la demanda semanal de productos y categorías mediante ventanas históricas de 24 semanas completas (6 meses) agrupando ventas diarias acumuladas.

- **Regularización L2:** Matriz diagonal [0.0, 0.25, 0.25, 0.25, 0.25] aplicados al intercepto y lags.
- **Ecuación de Inferencia:** $y_pred = \beta_0 + \beta_1 \cdot y_{t-4} + \beta_2 \cdot y_{t-3} + \beta_3 \cdot y_{t-2} + \beta_4 \cdot y_{t-1}$

1.2 Métricas de Entrenamiento y Artefacto Expuesto

El error residual del entrenamiento se evalúa mediante la **Raíz del Error Cuadrático Medio (RMSE)** sobre las filas históricas generadas (típicamente 20 muestras correspondientes a 24 semanas menos 4 lags temporales).

Estructura del Artefacto JSON Expuesto (`demanda.json`):

```
{
  "version": "ridge-lag4-v1",
  "trained_at": "2026-10-21T10:00:00Z",
  "source": "facturas_pagadas",
  "models": {
    "producto:123": {
      "coefficients": [0.5, 0.2, 0.15, 0.1, 0.05],
      "error": 2.34,
      "training_rows": 20,
      "history_weeks": 24
    }
  }
}
```

1.3 Métricas de Validación e Inferencia Calculadas

Métrica	Fórmula / Lógica de Cálculo	Propósito Operativo
Venta Proyectada	`Σ predicciones_diarias` (Horizonte)	Estimación total de demanda en el periodo.
Días Agotamiento	Día donde `demanda_acumulada ≥ stock_actual`	Alerta temprana de quiebre de stock.
Rotación Proyectada	`demanda_proyectada / inventario_promedio`	Medida de eficiencia del inventario.
Sugerencia Compra	`max(0, ceil(demanda + stock_min - stock_act))`	Recomendación automatizada de reabastecimiento.

1.4 Métricas de Latencia y Calidad del Sistema

Las llamadas entre el proxy PHP y el microservicio FastAPI están reguladas por timeouts estrictos para garantizar que la experiencia de usuario se mantenga fluida en el dashboard principal.

Operación	Componentes Clave	Timeout Configurado	Estimación Típica
Entrenamiento	Consulta SQL, Pydantic, Ridge Solve, Escritura JSON	30 segundos	2 - 5s (100 prod)
Inferencia (Dashboard)	Carga `demanda.json`, Multiplicación Matriz-Vector	12 segundos	100 - 300ms (Prod Ind.)
Retroalimentación	Validación Pydantic, Append atómico JSONL, `fsync`	8 segundos	< 50ms

1.5 Matriz de Confusión y Adaptación a Regresión

Debido a que el modelo es puramente de regresión cuantitativa, no calcula una matriz de confusión en su pipeline nativo. No obstante, para decisiones binarias de negocio (ej. *Comprar vs. No Comprar*), el sistema se parametriza formalmente bajo la siguiente estructura:

	Predicción: Stock Suficiente	**Predicción: Stock Insuficiente**
Real: Stock Suficiente	Verdaderos Positivos (VP)	Falsos Positivos (FP)
Real: Stock Insuficiente	Falsos Negativos (FN)	Verdaderos Negativos (VN)

1.6 Estados de Calidad y Niveles de Confianza

El microservicio expone estados explícitos en la API para prevenir mostrar datos engañosos a los operadores:

- `disponible` Modelo entrenado y listo para inferencia.
- `modelo_pendiente` Existen ventas históricas pero falta ejecutar el entrenamiento.
- `insuficiente_historial` Menos de 6 semanas con registros dentro de la ventana de 24 semanas.
- `sin_ventas` / `sin_datos` Ausencia total de transacciones o IDs no coincidentes.

Cálculo de Nivel de Confianza: `confianza = max(0.0, min(0.99, 1.0 - error / (media_ventas + 1.0)))`

2. Plan de Pruebas y Respuestas ante Fallos

2.1 Estrategia de Pruebas Implementadas

La suite de pruebas automatizadas está codificada en `test_analitica_predictiva.py` e incluye las siguientes verificaciones críticas:

- **Prueba de No Negatividad e Intervalos:** Valida que las predicciones sean `≥ 0` y contenidas dentro del rango del error residual.
- **Principio de Aislamiento de Datos:** Si faltan datos de inventario o stock mínimo, los KPIs retornan `None` para evitar decisiones con datos ficticios.
- **Protección de Artefacto:** Un reentrenamiento fallido por falta de datos no sobrescribe el archivo de modelo válido anterior.
- **Sugerencias por Canasta Frecuente:** Detecta parejas de productos comprados juntos en facturas (soporte `≥ 2`).

2.2 Respuestas Sistemáticas ante Fallos

> **Matriz de Mitigación de Fallos Operativos:**
>
> - **Fallo de Conectividad HTTP / Timeout:** PHP captura la excepción cURL y deshabilita controles en UI sin interrumpir la vista del usuario.
> - **Corrupción de Artefacto (JSON dañados):** El endpoint eleva `RuntimeError` notificando al usuario que debe forzar un reentrenamiento.
> - **Falta de Token / Desincronización:** FastAPI rechaza las consultas con HTTP 403 o HTTP 503 impidiendo accesos no autorizados.

2.3 Plan de Monitoreo y Umbrales de Alerta

Métrica Monitoreada	Umbral Advertencia	Umbral Crítico	Acción Correctiva
Latencia Dashboard (p95)	> 2.0s	> 5.0s	Optimizar consultas SQL a MariaDB.
Latencia Entrenamiento	> 20.0s	> 60.0s	Ajustar hilos/workers en FastAPI.
Tamaño `demanda.json`	> 10 MB	> 50 MB	Depurar/particionar historial de modelos.
Error HTTP (4xx / 5xx)	> 5%	> 15%	Revisar logs y sincronización de token.

3. Pruebas de Mantenimiento, Actualización y Recuperación

3.1 Protocolo de Mantenimiento Rutinario

- **Semanal:** Verificación del endpoint de salud mediante cURL, monitoreo del tamaño de `demanda.json` y rotación del log `feedback_analitica.jsonl` si supera 100 MB.
- **Mensual:** Reentrenamiento programado completo del modelo desde el panel administrativo y auditoría de la precisión histórica de predicciones.
- **Trimestral:** Actualización de dependencias Python (Pydantic, FastAPI, NumPy) en entorno de prueba previo a despliegue en producción.

3.2 Procedimiento de Actualización de Modelos

1. Desarrollo y ajuste del nuevo algoritmo en rama aislada (ej. `ridge-lag4-v2`).
2. Ejecución de suite completa de pruebas unitarias y de integración.
3. Generación de backup de seguridad del modelo activo.
4. Despliegue de código y reentrenamiento inmediato. Monitoreo intensivo de latencia por 24 horas.

3.3 Recuperación ante Desastres (DRP)

Escenario de Desastre	Efecto en el Sistema	Procedimiento de Recuperación
Pérdida de `demanda.json`	Estado pasa a *modelo_pendiente*	Restaurar backup diario o ejecutar reentrenamiento total desde la UI.
Pérdida de Feedback JSONL	Ajuste de recomendación vuelve a baseline	Restaurar log o permitir que el archivo se regenere con nuevas valoraciones.
Compromiso de Token Security	Bloqueo HTTP 403 en peticiones	Generar nuevo `ANALITICA_PREDICTIVA_TOKEN` mediante `secrets` y actualizar variables de entorno en Apache y Python.

> **Pruebas Canary / Despliegue Escalonado:**
> Para cambios mayores de arquitectura, el microservicio se despliega paralelamente en un puerto alternativo (ej. 8001). El proxy PHP deriva un 10% del tráfico para evaluar la tasa de errores antes del despliegue total.

4. Métricas de Éxito y KPIs del Sistema

Categoría	Indicador Clave (KPI)	Meta Objetivo	Estado / Implementación
Técnico	Disponibilidad del Servicio	≥ 99.5%	Monitoreado en Loopback local
Técnico	Latencia Inferencia (p95)	< 2.0 segundos	Optimizado vía Numpy mat-mult
Negocio	Adopción de Sugerencias de Compra	≥ 60% de usuarios	Registrado vía retroalimentación
Negocio	Reducción de Quiebres de Stock	≥ 20% vs. Baseline	Evaluación trimestral acumulada

5. Conclusiones y Roadmap de Mejoras

Hoja de Ruta Estratégica (Roadmap)

- **Corto Plazo (1-2 meses):** Validación temporal mediante *rolling window*, adición de métricas MAPE/MAE al reporte y circuit breaker en el proxy PHP.
- **Mediano Plazo (3-6 meses):** Migración del archivo JSONL de feedback a tabla relacional en MariaDB y panel Grafana para trazabilidad.
- **Largo Plazo (6-12 meses):** Evaluación de modelos avanzados (Prophet / LSTM) e implementación de pruebas A/B de algoritmos en paralelo.

Anexo: Diagrama de Arquitectura del Sistema

